

Algorithms

THIRD EDITION

IN C

Parts 1–4

FUNDAMENTALS
DATA STRUCTURES
SORTING
SEARCHING

ROBERT SEDGEWICK

Algorithms

THIRD EDITION

in C

PARTS 1–4

FUNDAMENTALS
DATA STRUCTURES
SORTING
SEARCHING

Robert Sedgewick
Princeton University



ADDISON-WESLEY

Boston • San Francisco • New York • Toronto • Montreal
London • Munich • Paris • Madrid
Capetown • Sydney • Tokyo • Singapore • Mexico City

Publishing Partner: Peter S. Gordon
Associate Editor: Deborah Lafferty
Cover Designer: Andre Kuzniarek
Production Editor: Amy Willcutt
Copy Editor: Lyn Dupre

The programs and applications presented in this book have been included for their instructional value. They have been tested with care, but are not guaranteed for any particular purpose. The publisher neither offers any warranties or representations, nor accepts any liabilities with respect to the programs or applications.

Library of Congress Cataloging-in-Publication Data

Sedgewick, Robert, 1946 –

Algorithms in C / Robert Sedgewick. — 3d ed.

720 p. 24 cm.

Includes bibliographical references and index.

Contents: v. 1, pts. 1–4. Fundamentals, data structures,
sorting, searching.

ISBN 0-201-31452-5

1. C (Computer program language) 2. Computer algorithms.

I. Title.

QA76.73.C15S43 1998

005.13'3—dc21

97-23418

CIP

Reproduced by Addison-Wesley from camera-ready copy supplied by the author.

Copyright © 1998 by Addison-Wesley Publishing Company, Inc.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the publisher. Printed in the United States of America.

Text printed on recycled and acid-free paper.

ISBN 0201314525

14 1516171819 CRS 08 07 06

14th Printing January 2006

Preface

THIS BOOK IS intended to survey the most important computer algorithms in use today, and to teach fundamental techniques to the growing number of people in need of knowing them. It can be used as a textbook for a second, third, or fourth course in computer science, after students have acquired basic programming skills and familiarity with computer systems, but before they have taken specialized courses in advanced areas of computer science or computer applications. The book also may be useful for self-study or as a reference for people engaged in the development of computer systems or applications programs, since it contains implementations of useful algorithms and detailed information on these algorithms' performance characteristics. The broad perspective taken makes the book an appropriate introduction to the field.

I have completely rewritten the text for this new edition, and I have added more than a thousand new exercises, more than a hundred new figures, and dozens of new programs. I have also added detailed commentary on all the figures and programs. This new material provides both coverage of new topics and fuller explanations of many of the classic algorithms. A new emphasis on abstract data types throughout the book makes the programs more broadly useful and relevant in modern object-oriented programming environments. People who have read old editions of the book will find a wealth of new information throughout; all readers will find a wealth of pedagogical material that provides effective access to essential concepts.

Due to the large amount of new material, we have split the new edition into two volumes (each about the size of the old edition) of which this is the first. This volume covers fundamental concepts, data structures, sorting algorithms, and searching algorithms; the second volume covers advanced algorithms and applications, building on the basic abstractions and methods developed here. Nearly all the material on fundamentals and data structures in this edition is new.

This book is not just for programmers and computer-science students. Nearly everyone who uses a computer wants it to run faster or to solve larger problems. The algorithms in this book represent a body of knowledge developed over the last 50 years that has become indispensable in the efficient use of the computer, for a broad variety of applications. From N -body simulation problems in physics to genetic-sequencing problems in molecular biology, the basic methods described here have become essential in scientific research; and from database systems to Internet search engines, they have become essential parts of modern software systems. As the scope of computer applications becomes more widespread, so grows the impact of many of the basic methods covered here. The goal of this book is to serve as a resource for students and professionals interested in knowing and making intelligent use of these fundamental algorithms as basic tools for whatever computer application they might undertake.

Scope

The book contains 16 chapters grouped into four major parts: fundamentals, data structures, sorting, and searching. The descriptions here are intended to give readers an understanding of the basic properties of as broad a range of fundamental algorithms as possible. Ingenious methods ranging from binomial queues to patricia tries are described, all related to basic paradigms at the heart of computer science. The second volume consists of four additional parts that cover strings, geometry, graphs, and advanced topics. My primary goal in developing these books has been to bring together the fundamental methods from these diverse areas, to provide access to the best methods known for solving problems by computer.

You will most appreciate the material in this book if you have had one or two previous courses in computer science or have had equivalent programming experience: one course in programming in a high-level language such as C, Java, or C++, and perhaps another course that teaches fundamental concepts of programming systems. This book is thus intended for anyone conversant with a modern programming language and with the basic features of modern computer systems. References that might help to fill in gaps in your background are suggested in the text.

Most of the mathematical material supporting the analytic results is self-contained (or is labeled as beyond the scope of this book), so little specific preparation in mathematics is required for the bulk of the book, although mathematical maturity is definitely helpful.

Use in the Curriculum

There is a great deal of flexibility in how the material here can be taught, depending on the taste of the instructor and the preparation of the students. The algorithms described here have found widespread use for years, and represent an essential body of knowledge for both the practicing programmer and the computer-science student. There is sufficient coverage of basic material for the book to be used for a course on data structures, and there is sufficient detail and coverage of advanced material for the book to be used for a course on algorithms. Some instructors may wish to emphasize implementations and practical concerns; others may wish to emphasize analysis and theoretical concepts.

A complete set of slide masters for use in lectures, sample programming assignments, interactive exercises for students, and other course materials may be found via the book's home page.

An elementary course on data structures and algorithms might emphasize the basic data structures in Part 2 and their use in the implementations in Parts 3 and 4. A course on design and analysis of algorithms might emphasize the fundamental material in Part 1 and Chapter 5, then study the ways in which the algorithms in Parts 3 and 4 achieve good asymptotic performance. A course on software engineering might omit the mathematical and advanced algorithmic material, and emphasize how to integrate the implementations given here into large programs or systems. A course on algorithms might take a survey approach and introduce concepts from all these areas.

Earlier editions of this book have been used in recent years at scores of colleges and universities around the world as a text for the second or third course in computer science and as supplemental reading for other courses. At Princeton, our experience has been that the breadth of coverage of material in this book provides our majors with an introduction to computer science that can be expanded upon in later courses on analysis of algorithms, systems programming and

theoretical computer science, while providing the growing group of students from other disciplines with a large set of techniques that these people can immediately put to good use.

The exercises—most of which are new to this edition—fall into several types. Some are intended to test understanding of material in the text, and simply ask readers to work through an example or to apply concepts described in the text. Others involve implementing and putting together the algorithms, or running empirical studies to compare variants of the algorithms and to learn their properties. Still others are a repository for important information at a level of detail that is not appropriate for the text. Reading and thinking about the exercises will pay dividends for every reader.

Algorithms of Practical Use

Anyone wanting to use a computer more effectively can use this book for reference or for self-study. People with programming experience can find information on specific topics throughout the book. To a large extent, you can read the individual chapters in the book independently of the others, although, in some cases, algorithms in one chapter make use of methods from a previous chapter.

The orientation of the book is to study algorithms likely to be of practical use. The book provides information about the tools of the trade to the point that readers can confidently implement, debug, and put to work algorithms to solve a problem or to provide functionality in an application. Full implementations of the methods discussed are included, as are descriptions of the operations of these programs on a consistent set of examples. Because we work with real code, rather than write pseudo-code, the programs can be put to practical use quickly. Program listings are available from the book's home page.

Indeed, one practical application of the algorithms has been to produce the hundreds of figures throughout the book. Many algorithms are brought to light on an intuitive level through the visual dimension provided by these figures.

Characteristics of the algorithms and of the situations in which they might be useful are discussed in detail. Although not emphasized, connections to the analysis of algorithms and theoretical computer science are developed in context. When appropriate, empirical and

analytic results are presented to illustrate why certain algorithms are preferred. When interesting, the relationship of the practical algorithms being discussed to purely theoretical results is described. Specific information on performance characteristics of algorithms and implementations is synthesized, encapsulated, and discussed throughout the book.

Programming Language

The programming language used for all of the implementations is C. Any particular language has advantages and disadvantages; we use C because it is widely available and provides the features needed for our implementations. The programs can be translated easily to other modern programming languages, since relatively few constructs are unique to C. We use standard C idioms when appropriate, but this book is not intended to be a reference work on C programming.

There are many new programs in this edition, and many of the old ones have been reworked, primarily to make them more readily useful as abstract-data-type implementations. Extensive comparative empirical tests on the programs are discussed throughout the text.

Previous editions of the book have presented basic programs in Pascal, C++, and Modula-3. This code is available through the book home page on the web; code for new programs and code in new languages such as Java will be added as appropriate.

A goal of this book is to present the algorithms in as simple and direct a form as possible. The style is consistent whenever possible, so that programs that are similar look similar. For many of the algorithms in this book, the similarities hold regardless of the language: Quicksort is quicksort (to pick one prominent example), whether expressed in Algol-60, Basic, Fortran, Smalltalk, Ada, Pascal, C, PostScript, Java, or countless other programming languages and environments where it has proved to be an effective sorting method.

We strive for elegant, compact, and portable implementations, but we take the point of view that efficiency matters, so we try to be aware of the performance characteristics of our code at all stages of development. Chapter 1 constitutes a detailed example of this approach to developing efficient C implementations of our algorithms, and sets the stage for the rest of the book.

Acknowledgments

Many people gave me helpful feedback on earlier versions of this book. In particular, hundreds of students at Princeton and Brown have suffered through preliminary drafts over the years. Special thanks are due to Trina Avery and Tom Freeman for their help in producing the first edition; to Janet Incerpi for her creativity and ingenuity in persuading our early and primitive digital computerized typesetting hardware and software to produce the first edition; to Marc Brown for his part in the algorithm visualization research that was the genesis of so many of the figures in the book; and to Dave Hanson for his willingness to answer all of my questions about C. I would also like to thank the many readers who have provided me with detailed comments about various editions, including Guy Almes, Jon Bentley, Marc Brown, Jay Gischer, Allan Heydon, Kennedy Lemke, Udi Manber, Dana Richards, John Reif, M. Rosenfeld, Stephen Seidman, Michael Quinn, and William Ward.

To produce this new edition, I have had the pleasure of working with Peter Gordon and Debbie Lafferty at Addison-Wesley, who have patiently shepherded this project as it has evolved from a standard update to a massive rewrite. It has also been my pleasure to work with several other members of the professional staff at Addison-Wesley. The nature of this project made the book a somewhat unusual challenge for many of them, and I much appreciate their forbearance.

I have gained two new mentors in writing this book, and particularly want to express my appreciation to them. First, Steve Summit carefully checked early versions of the manuscript on a technical level, and provided me with literally thousands of detailed comments, particularly on the programs. Steve clearly understood my goal of providing elegant, efficient, and effective implementations, and his comments not only helped me to provide a measure of consistency across the implementations, but also helped me to improve many of them substantially. Second, Lyn Dupre also provided me with thousands of detailed comments on the manuscript, which were invaluable in helping me not only to correct and avoid grammatical errors, but also—more important—to find a consistent and coherent writing style that helps bind together the daunting mass of technical material here. I am extremely grateful

for the opportunity to learn from Steve and Lyn—their input was vital in the development of this book.

Much of what I have written here I have learned from the teaching and writings of Don Knuth, my advisor at Stanford. Although Don had no direct influence on this work, his presence may be felt in the book, for it was he who put the study of algorithms on the scientific footing that makes a work such as this possible. My friend and colleague Philippe Flajolet, who has been a major force in the development of the analysis of algorithms as a mature research area, has had a similar influence on this work.

I am deeply thankful for the support of Princeton University, Brown University, and the Institut National de Recherche en Informatique et Automatique (INRIA), where I did most of the work on the book; and of the Institute for Defense Analyses and the Xerox Palo Alto Research Center, where I did some work on the book while visiting. Many parts of the book are dependent on research that has been generously supported by the National Science Foundation and the Office of Naval Research. Finally, I thank Bill Bowen, Aaron Lemonick, and Neil Rudenstine for their support in building an academic environment at Princeton in which I was able to prepare this book, despite my numerous other responsibilities.

Robert Sedgewick
Marly-le-Roi, France, February, 1983
Princeton, New Jersey, January, 1990
Jamestown, Rhode Island, August, 1997

*To Adam, Andrew, Brett, Robbie,
and especially Linda*

Notes on Exercises

Classifying exercises is an activity fraught with peril, because readers of a book such as this come to the material with various levels of knowledge and experience. Nonetheless, guidance is appropriate, so many of the exercises carry one of four annotations, to help you decide how to approach them.

Exercises that *test your understanding* of the material are marked with an open triangle, as follows:

- ▷ 9.57 Give the binomial queue that results when the keys EASY QUESTION are inserted into an initially empty binomial queue.

Most often, such exercises relate directly to examples in the text. They should present no special difficulty, but working them might teach you a fact or concept that may have eluded you when you read the text.

Exercises that *add new and thought-provoking* information to the material are marked with an open circle, as follows:

- 14.20 Write a program that inserts N random integers into a table of size $N/100$ using separate chaining, then finds the length of the shortest and longest lists, for $N = 10^3, 10^4, 10^5$, and 10^6 .

Such exercises encourage you to think about an important concept that is related to the material in the text, or to answer a question that may have occurred to you when you read the text. You may find it worthwhile to read these exercises, even if you do not have the time to work them through.

Exercises that are intended to *challenge you* are marked with a black dot, as follows:

- 8.46 Suppose that mergesort is implemented to split the file at a *random* position, rather than exactly in the middle. How many comparisons are used by such a method to sort N elements, on the average?

Such exercises may require a substantial amount of time to complete, depending upon your experience. Generally, the most productive approach is to work on them in a few different sittings.

A few exercises that are *extremely difficult* (by comparison with most others) are marked with two black dots, as follows:

- 15.29 Prove that the height of a trie built from N random bit-strings is about $2 \lg N$.

These exercises are similar to questions that might be addressed in the research literature, but the material in the book may prepare you to enjoy trying to solve them (and perhaps succeeding).

The annotations are intended to be neutral with respect to your programming and mathematical ability. Those exercises that require expertise in programming or in mathematical analysis are self-evident. All readers are encouraged to test their understanding of the algorithms by implementing them. Still, an exercise such as this one is straightforward for a practicing programmer or a student in a programming course, but may require substantial work for someone who has not recently programmed:

1.23 Modify Program 1.4 to generate random pairs of integers between 0 and $N - 1$ instead of reading them from standard input, and to loop until $N - 1$ *union* operations have been performed. Run your program for $N = 10^3, 10^4, 10^5$, and 10^6 and print out the total number of edges generated for each value of N .

In a similar vein, all readers are encouraged to strive to appreciate the analytic underpinnings of our knowledge about properties of algorithms. Still, an exercise such as this one is straightforward for a scientist or a student in a discrete mathematics course, but may require substantial work for someone who has not recently done mathematical analysis:

1.13 Compute the *average* distance from a node to the root in a worst-case tree of 2^n nodes built by the weighted quick-union algorithm.

There are far too many exercises for you to read and assimilate them all; my hope is that there are enough exercises here to stimulate you to strive to come to a broader understanding on the topics that interest you than you can glean by simply reading the text.

Contents

Fundamentals

Chapter 1. Introduction 3

- 1.1 Algorithms · 4
- 1.2 A Sample Problem—Connectivity · 6
- 1.3 Union-Find Algorithms · 11
- 1.4 Perspective · 22
- 1.5 Summary of Topics · 23

Chapter 2. Principles of Algorithm Analysis 27

- 2.1 Implementation and Empirical Analysis · 28
- 2.2 Analysis of Algorithms · 33
- 2.3 Growth of Functions · 36
- 2.4 Big-Oh notation · 44
- 2.5 Basic Recurrences · 49
- 2.6 Examples of Algorithm Analysis · 53
- 2.7 Guarantees, Predictions, and Limitations · 59

Data Structures

Chapter 3. Elementary Data Structures	69
3.1 Building Blocks · 70	
3.2 Arrays · 82	
3.3 Linked Lists · 90	
3.4 Elementary List Processing · 96	
3.5 Memory Allocation for Lists · 105	
3.6 Strings · 109	
3.7 Compound Data Structures · 115	
Chapter 4. Abstract Data Types	127
4.1 Abstract Objects and Collections of Objects · 131	
4.2 Pushdown Stack ADT · 135	
4.3 Examples of Stack ADT Clients · 138	
4.4 Stack ADT Implementations · 144	
4.5 Creation of a New ADT · 149	
4.6 FIFO Queues and Generalized Queues · 153	
4.7 Duplicate and Index Items · 161	
4.8 First-Class ADTs · 165	
4.9 Application-Based ADT Example · 178	
4.10 Perspective · 184	
Chapter 5. Recursion and Trees	187
5.1 Recursive Algorithms · 188	
5.2 Divide and Conquer · 196	
5.3 Dynamic Programming · 208	
5.4 Trees · 216	
5.5 Mathematical Properties of Trees · 226	
5.6 Tree Traversal · 230	
5.7 Recursive Binary-Tree Algorithms · 235	
5.8 Graph Traversal · 241	
5.9 Perspective · 247	

Sorting

Chapter 6. Elementary Sorting Methods **253**

- 6.1 Rules of the Game · 255
- 6.2 Selection Sort · 261
- 6.3 Insertion Sort · 262
- 6.4 Bubble Sort · 265
- 6.5 Performance Characteristics of Elementary Sorts · 267
- 6.6 Shellsort · 273
- 6.7 Sorting Other Types of Data · 281
- 6.8 Index and Pointer Sorting · 287
- 6.9 Sorting of Linked Lists · 294
- 6.10 Key-Indexed Counting · 298

Chapter 7. Quicksort **303**

- 7.1 The Basic Algorithm · 304
- 7.2 Performance Characteristics of Quicksort · 309
- 7.3 Stack Size · 313
- 7.4 Small Subfiles · 316
- 7.5 Median-of-Three Partitioning · 319
- 7.6 Duplicate Keys · 324
- 7.7 Strings and Vectors · 327
- 7.8 Selection · 329

Chapter 8. Merging and Mergesort **335**

- 8.1 Two-Way Merging · 336
- 8.2 Abstract In-place Merge · 339
- 8.3 Top-Down Mergesort · 341
- 8.4 Improvements to the Basic Algorithm · 344
- 8.5 Bottom-Up Mergesort · 347
- 8.6 Performance Characteristics of Mergesort · 351
- 8.7 Linked-List Implementations of Mergesort · 354
- 8.8 Recursion Revisited · 357

Chapter 9. Priority Queues and Heapsort **361**

- 9.1 Elementary Implementations · 365
- 9.2 Heap Data Structure · 368

- 9.3 Algorithms on Heaps · 371
- 9.4 Heapsort · 376
- 9.5 Priority-Queue ADT · 383
- 9.6 Priority Queues for Index Items · 389
- 9.7 Binomial Queues · 392

Chapter 10. Radix Sorting **403**

- 10.1 Bits, Bytes, and Words · 405
- 10.2 Binary Quicksort · 409
- 10.3 MSD Radix Sort · 413
- 10.4 Three-Way Radix Quicksort · 421
- 10.5 LSD Radix Sort · 425
- 10.6 Performance Characteristics of Radix Sorts · 428
- 10.7 Sublinear-Time Sorts · 433

Chapter 11. Special-Purpose Sorts **439**

- 11.1 Batcher's Odd-Even Mergesort · 441
- 11.2 Sorting Networks · 446
- 11.3 External Sorting · 454
- 11.4 Sort-Merge Implementations · 460
- 11.5 Parallel Sort/Merge · 466

Searching

Chapter 12. Symbol Tables and BSTs **477**

- 12.1 Symbol-Table Abstract Data Type · 479
- 12.2 Key-Indexed Search · 485
- 12.3 Sequential Search · 489
- 12.4 Binary Search · 497
- 12.5 Binary Search Trees (BSTs) · 502
- 12.6 Performance Characteristics of BSTs · 508
- 12.7 Index Implementations with Symbol Tables · 511
- 12.8 Insertion at the Root in BSTs · 516
- 12.9 BST Implementations of Other ADT Functions · 519

Chapter 13. Balanced Trees **529**

- 13.1 Randomized BSTs · 533
- 13.2 Splay BSTs · 540
- 13.3 Top-Down 2-3-4 Trees · 546
- 13.4 Red-Black Trees · 551
- 13.5 Skip Lists · 561
- 13.6 Performance Characteristics · 569

Chapter 14. Hashing **573**

- 14.1 Hash Functions · 574
- 14.2 Separate Chaining · 583
- 14.3 Linear Probing · 588
- 14.4 Double Hashing · 594
- 14.5 Dynamic Hash Tables · 599
- 14.6 Perspective · 603

Chapter 15. Radix Search **609**

- 15.1 Digital Search Trees · 610
- 15.2 Tries · 614
- 15.3 Patricia Tries · 623
- 15.4 Multiway Tries and TSTs · 632
- 15.5 Text String Index Algorithms · 648

Chapter 16. External Searching **655**

- 16.1 Rules of the Game · 657
- 16.2 Indexed Sequential Access · 660
- 16.3 B Trees · 662
- 16.4 Extendible Hashing · 676
- 16.5 Perspective · 688

Index **693**

P A R T
O N E

Fundamentals

CHAPTER ONE

Introduction

THE OBJECTIVE OF this book is to study a broad variety of important and useful *algorithms*: methods for solving problems that are suited for computer implementation. We shall deal with many different areas of application, always concentrating on fundamental algorithms that are important to know and interesting to study. We shall spend enough time on each algorithm to understand its essential characteristics and to respect its subtleties. Our goal is to learn a large number of the most important algorithms used on computers today, well enough to be able to use and appreciate them.

The strategy that we use for understanding the programs presented in this book is to implement and test them, to experiment with their variants, to discuss their operation on small examples, and to try them out on larger examples similar to what we might encounter in practice. We shall use the C programming language to describe the algorithms, thus providing useful implementations at the same time. Our programs have a uniform style that is amenable to translation into other modern programming languages, as well.

We also pay careful attention to performance characteristics of our algorithms, to help us develop improved versions, compare different algorithms for the same task, and predict or guarantee performance for large problems. Understanding how the algorithms perform might require experimentation or mathematical analysis or both. We consider detailed information for many of the most important algorithms, developing analytic results directly when feasible, or calling on results from the research literature when necessary.

To illustrate our general approach to developing algorithmic solutions, we consider in this chapter a detailed example comprising a number of algorithms that solve a particular problem. The problem that we consider is not a toy problem; it is a fundamental computational task, and the solution that we develop is of use in a variety of applications. We start with a simple solution, then seek to understand that solution's performance characteristics, which help us to see how to improve the algorithm. After a few iterations of this process, we come to an efficient and useful algorithm for solving the problem. This prototypical example sets the stage for our use of the same general methodology throughout the book.

We conclude the chapter with a short discussion of the contents of the book, including brief descriptions of what the major parts of the book are and how they relate to one another.

1.1 Algorithms

When we write a computer program, we are generally implementing a method that has been devised previously to solve some problem. This method is often independent of the particular computer to be used—it is likely to be equally appropriate for many computers and many computer languages. It is the method, rather than the computer program itself, that we must study to learn how the problem is being attacked. The term *algorithm* is used in computer science to describe a problem-solving method suitable for implementation as a computer program. Algorithms are the stuff of computer science: They are central objects of study in many, if not most, areas of the field.

Most algorithms of interest involve methods of organizing the data involved in the computation. Objects created in this way are called *data structures*, and they also are central objects of study in computer science. Thus, algorithms and data structures go hand in hand. In this book we take the view that data structures exist as the byproducts or end products of algorithms, and thus that we must study them in order to understand the algorithms. Simple algorithms can give rise to complicated data structures and, conversely, complicated algorithms can use simple data structures. We shall study the properties of many data structures in this book; indeed, the book might well have been called *Algorithms and Data Structures in C*.

When we use a computer to help us solve a problem, we typically are faced with a number of possible different approaches. For small problems, it hardly matters which approach we use, as long as we have one that solves the problem correctly. For huge problems (or applications where we need to solve huge numbers of small problems), however, we quickly become motivated to devise methods that use time or space as efficiently as possible.

The primary reason for us to learn about algorithm design is that this discipline gives us the potential to reap huge savings, even to the point of making it possible to do tasks that would otherwise be impossible. In an application where we are processing millions of objects, it is not unusual to be able to make a program millions of times faster by using a well-designed algorithm. We shall see such an example in Section 1.2 and on numerous other occasions throughout the book. By contrast, investing additional money or time to buy and install a new computer holds the potential for speeding up a program by perhaps a factor of only 10 or 100. Careful algorithm design is an extremely effective part of the process of solving a huge problem, whatever the applications area.

When a huge or complex computer program is to be developed, a great deal of effort must go into understanding and defining the problem to be solved, managing its complexity, and decomposing it into smaller subtasks that can be implemented easily. Often, many of the algorithms required after the decomposition are trivial to implement. In most cases, however, there are a few algorithms whose choice is critical because most of the system resources will be spent running those algorithms. Those are the types of algorithms on which we concentrate in this book. We shall study a variety of fundamental algorithms that are useful for solving huge problems in a broad variety of applications areas.

The sharing of programs in computer systems is becoming more widespread, so, although we might expect to be *using* a large fraction of the algorithms in this book, we also might expect to have to *implement* only a smaller fraction of them. However, implementing simple versions of basic algorithms helps us to understand them better and thus to use advanced versions more effectively. More important, the opportunity to reimplement basic algorithms arises frequently. The primary reason to do so is that we are faced, all too often, with com-

pletely new computing environments (hardware and software) with new features that old implementations may not use to best advantage. In other words, we often implement basic algorithms tailored to our problem, rather than depending on a system routine, to make our solutions more portable and longer lasting. Another common reason to reimplement basic algorithms is that mechanisms for sharing software on many computer systems are not always sufficiently powerful to allow us to tailor standard programs to perform effectively on specific tasks (or it may not be convenient to do so), so it is sometimes easier to do a new implementation.

Computer programs are often overoptimized. It may not be worthwhile to take pains to ensure that an implementation of a particular algorithm is the most efficient possible unless the algorithm is to be used for an enormous task or is to be used many times. Otherwise, a careful, relatively simple implementation will suffice: We can have some confidence that it will work, and it is likely to run perhaps five or 10 times slower at worst than the best possible version, which means that it may run for an extra few seconds. By contrast, the proper choice of algorithm in the first place can make a difference of a factor of 100 or 1000 or more, which might translate to minutes, hours, or even more in running time. In this book, we concentrate on the simplest reasonable implementations of the best algorithms.

The choice of the best algorithm for a particular task can be a complicated process, perhaps involving sophisticated mathematical analysis. The branch of computer science that comprises the study of such questions is called *analysis of algorithms*. Many of the algorithms that we study have been shown through analysis to have excellent performance; others are simply known to work well through experience. Our primary goal is to learn reasonable algorithms for important tasks, yet we shall also pay careful attention to comparative performance of the methods. We should not use an algorithm without having an idea of what resources it might consume, and we strive to be aware of how our algorithms might be expected to perform.

1.2 A Sample Problem: Connectivity

Suppose that we are given a sequence of pairs of integers, where each integer represents an object of some type and we are to interpret the

pair p - q as meaning “ p is connected to q .” We assume the relation “is connected to” to be transitive: If p is connected to q , and q is connected to r , then p is connected to r . Our goal is to write a program to filter out extraneous pairs from the set: When the program inputs a pair p - q , it should output the pair only if the pairs it has seen to that point *do not* imply that p is connected to q . If the previous pairs do imply that p is connected to q , then the program should ignore p - q and should proceed to input the next pair. Figure 1.1 gives an example of this process.

Our problem is to devise a program that can remember sufficient information about the pairs it has seen to be able to decide whether or not a new pair of objects is connected. Informally, we refer to the task of designing such a method as the *connectivity problem*. This problem arises in a number of important applications. We briefly consider three examples here to indicate the fundamental nature of the problem.

For example, the integers might represent computers in a large network, and the pairs might represent connections in the network. Then, our program might be used to determine whether we need to establish a new direct connection for p and q to be able to communicate, or whether we could use existing connections to set up a communications path. In this kind of application, we might need to process millions of points and billions of connections, or more. As we shall see, it would be impossible to solve the problem for such an application without an efficient algorithm.

Similarly, the integers might represent contact points in an electrical network, and the pairs might represent wires connecting the points. In this case, we could use our program to find a way to connect all the points without any extraneous connections, if that is possible. There is no guarantee that the edges in the list will suffice to connect all the points—indeed, we shall soon see that determining whether or not they will could be a prime application of our program.

Figure 1.2 illustrates these two types of applications in a larger example. Examination of this figure gives us an appreciation for the difficulty of the connectivity problem: How can we arrange to tell quickly whether *any* given two points in such a network are connected?

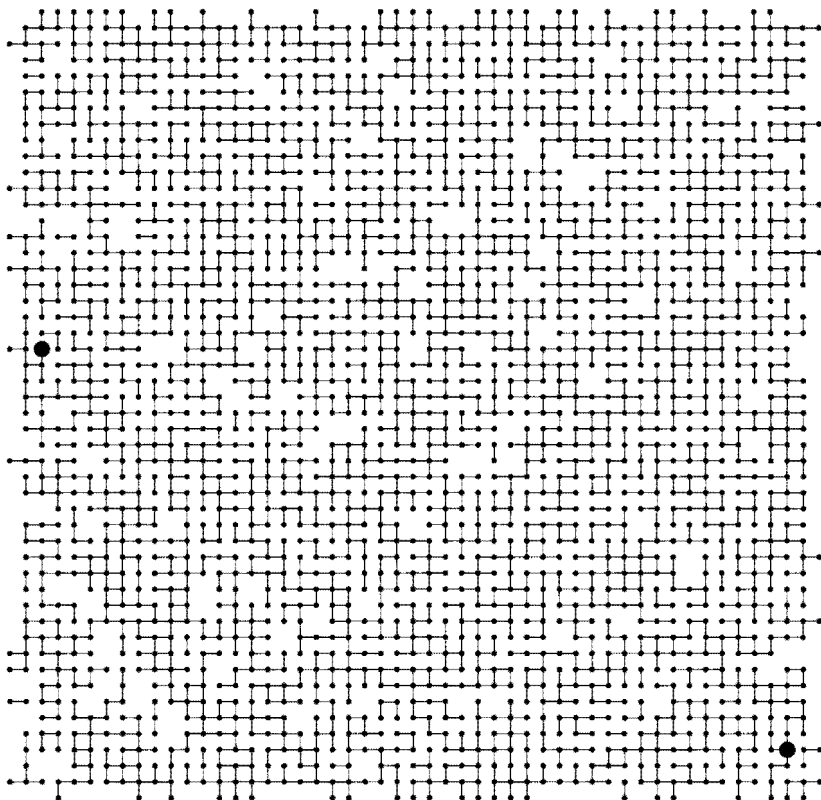
Still another example arises in certain programming environments where it is possible to declare two variable names as equivalent. The problem is to be able to determine whether two given names are

3-4	3-4	
4-9	4-9	
8-0	8-0	
2-3	2-3	
5-6	5-6	
2-9		2-3-4-9
5-9	5-9	
7-3	7-3	
4-8	4-8	
5-6		5-6
0-2		0-8-4-3-2
6-1	6-1	

Figure 1.1
Connectivity example
Given a sequence of pairs of integers representing connections between objects (left), the task of a connectivity algorithm is to output those pairs that provide new connections (center). For example, the pair 2-9 is not part of the output because the connection 2-3-4-9 is implied by previous connections (this evidence is shown at right).

Figure 1.2
A large connectivity example

The objects in a connectivity problem might represent connection points, and the pairs might be connections between them, as indicated in this idealized example that might represent wires connecting buildings in a city or components on a computer chip. This graphical representation makes it possible for a human to spot nodes that are not connected, but the algorithm has to work with only the pairs of integers that it is given. Are the two nodes marked with the large black dots connected?



equivalent, after a sequence of such declarations. This application is an early one that motivated the development of several of the algorithms that we are about to consider. It directly relates our problem to a simple abstraction that provides us with a way to make our algorithms useful for a wide variety of applications, as we shall see.

Applications such as the variable-name-equivalence problem described in the previous paragraph require that we associate an integer with each distinct variable name. This association is also implicit in the network-connection and circuit-connection applications that we have described. We shall be considering a host of algorithms in Chapters 10 through 16 that can provide this association in an efficient manner. Thus, we can assume in this chapter, without loss of generality, that we have N objects with integer names, from 0 to $N - 1$.

We are asking for a program that does a specific and well-defined task. There are many other related problems that we might want to have solved, as well. One of the first tasks that we face in developing an algorithm is to be sure that we have specified the *problem* in a reasonable manner. The more we require of an algorithm, the more time and space we may expect it to need to finish the task. It is impossible to quantify this relationship a priori, and we often modify a problem specification on finding that it is difficult or expensive to solve, or, in happy circumstances, on finding that an algorithm can provide information more useful than was called for in the original specification.

For example, our connectivity-problem specification requires only that our program somehow know whether or not any given pair p - q is connected, and not that it be able to demonstrate any or all ways to connect that pair. Adding a requirement for such a specification makes the problem more difficult, and would lead us to a different family of algorithms, which we consider briefly in Chapter 5 and in detail in Part 7.

The specifications mentioned in the previous paragraph ask us for *more* information than our original one did; we could also ask for *less* information. For example, we might simply want to be able to answer the question: “Are the M connections sufficient to connect together all N objects?” This problem illustrates that, to develop efficient algorithms, we often need to do high-level reasoning about the abstract objects that we are processing. In this case, a fundamental result from graph theory implies that all N objects are connected if and only if the number of pairs output by the connectivity algorithm is precisely $N - 1$ (see Section 5.4). In other words, a connectivity algorithm will never output more than $N - 1$ pairs, because, once it has output $N - 1$ pairs, any pair that it encounters from that point on will be connected. Accordingly, we can get a program that answers the yes–no question just posed by changing a program that solves the connectivity problem to one that increments a counter, rather than writing out each pair that was not previously connected, answering “yes” when the counter reaches $N - 1$ and “no” if it never does. This question is but one example of a host of questions that we might wish to answer regarding connectivity. The set of pairs in the input is called a *graph*, and the set of pairs output is called a *spanning tree* for

that graph, which connects all the objects. We consider properties of graphs, spanning trees, and all manner of related algorithms in Part 7.

It is worthwhile to try to identify the fundamental operations that we will be performing, and so to make any algorithm that we develop for the connectivity task useful for a variety of similar tasks. Specifically, each time that we get a new pair, we have first to determine whether it represents a new connection, then to incorporate the information that the connection has been seen into its understanding about the connectivity of the objects such that it can check connections to be seen in the future. We encapsulate these two tasks as *abstract operations* by considering the integer input values to represent elements in abstract sets, and then design algorithms and data structures that can

- *Find* the set containing a given item.
- Replace the sets containing two given items by their *union*.

Organizing our algorithms in terms of these abstract operations does not seem to foreclose any options in solving the connectivity problem, and the operations may be useful for solving other problems. Developing ever more powerful layers of abstraction is an essential process in computer science in general and in algorithm design in particular, and we shall turn to it on numerous occasions throughout this book. In this chapter, we use abstract thinking in an informal way to guide us in designing programs to solve the connectivity problem; in Chapter 4, we shall see how to encapsulate abstractions in C code.

The connectivity problem is easily solved in terms of the *find* and *union* abstract operations. After reading a new pair p - q from the input, we perform a *find* operation for each member of the pair. If the members of the pair are in the same set, we move on to the next pair; if they are not, we do a *union* operation and write out the pair. The sets represent *connected components*: subsets of the objects with the property that any two objects in a given component are connected. This approach reduces the development of an algorithmic solution for connectivity to the tasks of defining a data structure representing the sets and developing *union* and *find* algorithms that efficiently use that data structure.

There are many possible ways to represent and process abstract sets, which we consider in more detail in Chapter 4. In this chapter, our focus is on finding a representation that can support efficiently

the *union* and *find* operations that we see in solving the connectivity problem.

Exercises

1.1 Give the output that a connectivity algorithm should produce when given the input 0-2, 1-4, 2-5, 3-6, 0-4, 6-0, and 1-3.

1.2 List all the different ways to connect two different objects for the example in Figure 1.1.

1.3 Describe a simple method for counting the number of sets remaining after using the *union* and *find* operations to solve the connectivity problem as described in the text.

1.3 Union-Find Algorithms

The first step in the process of developing an efficient algorithm to solve a given problem is to *implement a simple algorithm that solves the problem*. If we need to solve a few particular problem instances that turn out to be easy, then the simple implementation may finish the job for us. If a more sophisticated algorithm is called for, then the simple implementation provides us with a correctness check for small cases and a baseline for evaluating performance characteristics. We always care about efficiency, but our primary concern in developing the first program that we write to solve a problem is to make sure that the program is a *correct* solution to the problem.

The first idea that might come to mind is somehow to save all the input pairs, then to write a function to pass through them to try to discover whether the next pair of objects is connected. We shall use a different approach. First, the number of pairs might be sufficiently large to preclude our saving them all in memory in practical applications. Second, and more to the point, no simple method immediately suggests itself for determining whether two objects are connected from the set of all the connections, even if we could save them all! We consider a basic method that takes this approach in Chapter 5, but the methods that we shall consider in this chapter are simpler, because they solve a less difficult problem, and are more efficient, because they do not require saving all the pairs. They all use an array of integers—one corresponding to each object—to hold the requisite information to be able to implement *union* and *find*.

p \ q	0	1	2	3	4	5	6	7	8	9
3 4	0	1	2	4	4	5	6	7	8	9
4 9	0	1	2	9	9	5	6	7	8	9
8 0	0	1	2	9	9	5	6	7	0	9
2 3	0	1	9	9	9	5	6	7	0	9
5 6	0	1	9	9	9	6	6	7	0	9
2 9	0	1	9	9	9	6	6	7	0	9
5 9	0	1	9	9	9	9	9	7	0	9
7 3	0	1	9	9	9	9	9	9	0	9
4 8	0	1	0	0	0	0	0	0	0	0
5 6	0	1	0	0	0	0	0	0	0	0
0 2	0	1	0	0	0	0	0	0	0	0
6 1	1	1	1	1	1	1	1	1	1	1

Figure 1.3
Example of quick find (slow union)

This sequence depicts the contents of the *id* array after each of the pairs at left is processed by the quick-find algorithm (Program 1.1). Shaded entries are those that change for the union operation. When we process the pair *p q*, we change all entries with the value *id[p]* to have the value *id[q]*.

Program 1.1 Quick-find solution to connectivity problem

This program reads a sequence of pairs of nonnegative integers less than N from standard input (interpreting the pair $p\ q$ to mean “connect object p to object q ”) and prints out pairs representing objects that are not yet connected. It maintains an array `id` that has an entry for each object, with the property that `id[p]` and `id[q]` are equal if and only if p and q are connected. For simplicity, we define N as a compile-time constant. Alternatively, we could take it from the input and allocate the `id` array dynamically (see Section 3.2).

```
#include <stdio.h>
#define N 10000
main()
{ int i, p, q, t, id[N];
  for (i = 0; i < N; i++) id[i] = i;
  while (scanf("%d %d\n", &p, &q) == 2)
  {
    if (id[p] == id[q]) continue;
    for (t = id[p], i = 0; i < N; i++)
      if (id[i] == t) id[i] = id[q];
    printf(" %d %d\n", p, q);
  }
}
```

Arrays are elementary data structures that we shall discuss in detail in Section 3.2. Here, we use them in their simplest form: we declare that we expect to use, say, 1000 integers, by writing `a[1000]`; then we refer to the i th integer in the array by writing `a[i]` for $0 \leq i < 1000$.

Program 1.1 is an implementation of a simple algorithm called the *quick-find algorithm* that solves the connectivity problem. The basis of this algorithm is an array of integers with the property that p and q are connected if and only if the p th and q th array entries are equal. We initialize the i th array entry to i for $0 \leq i < N$. To implement the *union* operation for p and q , we go through the array, changing all the entries with the same name as p to have the same name as q . This choice is arbitrary—we could have decided to change all the entries with the same name as q to have the same name as p .

Figure 1.3 shows the changes to the array for the *union* operations in the example in Figure 1.1. To implement *find*, we just test the indicated array entries for equality—hence the name *quick find*. The *union* operation, on the other hand, involves scanning through the whole array for each input pair.

Property 1.1 *The quick-find algorithm executes at least MN instructions to solve a connectivity problem with N objects that involves M union operations.*

For each of the M *union* operations, we iterate the for loop N times. Each iteration requires at least one instruction (if only to check whether the loop is finished). ■

We can execute tens or hundreds of millions of instructions per second on modern computers, so this cost is not noticeable if M and N are small, but we also might find ourselves with millions of objects and billions of input pairs to process in a modern application. The inescapable conclusion is that we cannot feasibly solve such a problem using the quick-find algorithm (see Exercise 1.10). We consider the process of quantifying such a conclusion precisely in Chapter 2.

Figure 1.4 shows a graphical representation of Figure 1.3. We may think of some of the objects as representing the set to which they belong, and all of the other objects as pointing to the representative in their set. The reason for moving to this graphical representation of the array will become clear soon. Observe that the connections between objects in this representation are *not* necessarily the same as the connections in the input pairs—they are the information that the algorithm chooses to remember to be able to know whether future pairs are connected.

The next algorithm that we consider is a complementary method called the *quick-union algorithm*. It is based on the same data structure—an array indexed by object names—but it uses a different interpretation of the values that leads to more complex abstract structures. Each object points to another object in the same set, in a structure with no cycles. To determine whether two objects are in the same set, we follow pointers for each until we reach an object that points to itself. The objects are in the same set if and only if this process leads them to the same object. If they are not in the same set, we wind up at different objects (which point to themselves). To form

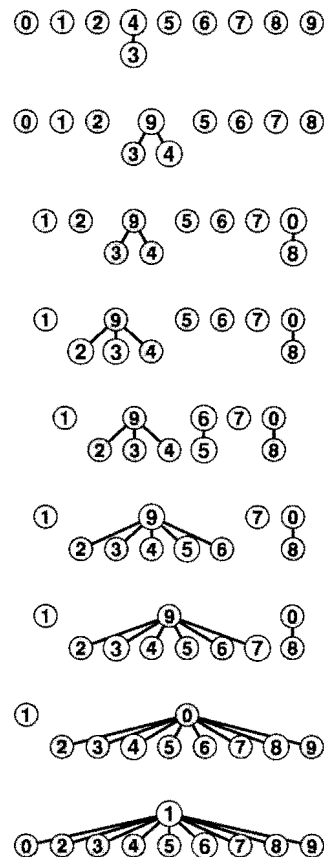


Figure 1.4
Tree representation of quick find

This figure depicts graphical representations for the example in Figure 1.3. The connections in these figures do not necessarily represent the connections in the input. For example, the structure at the bottom has the connection 1-7, which is not in the input, but which is made because of the string of connections 7-3-4-9-5-6-1.

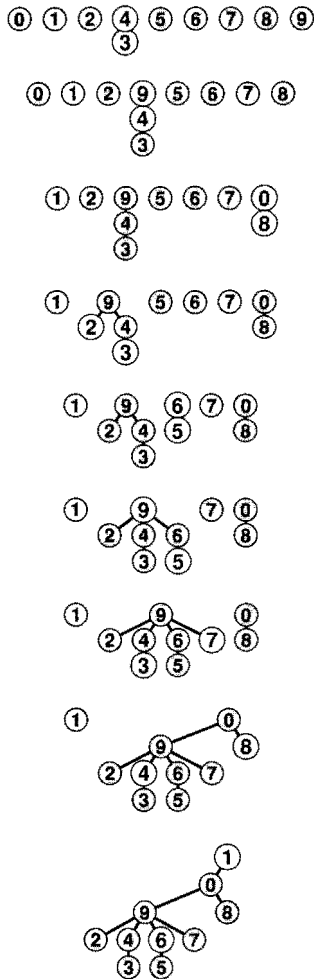


Figure 1.5
Tree representation of quick union

This figure is a graphical representation of the example in Figure 1.3. We draw a line from object i to object $\text{id}[i]$.

the union, then we just link one to the other to perform the *union* operation; hence the name *quick-union*.

Figure 1.5 shows the graphical representation that corresponds to Figure 1.4 for the operation of the quick-union algorithm on the example of Figure 1.1, and Figure 1.6 shows the corresponding changes to the `id` array. The graphical representation of the data structure makes it relatively easy to understand the operation of the algorithm—input pairs that are known to be connected in the data are also connected to one another in the data structure. As mentioned previously, it is important to note at the outset that the connections in the data structure are not necessarily the same as the connections in the application implied by the input pairs; rather, they are constructed by the algorithm to facilitate efficient implementation of *union* and *find*.

The connected components depicted in Figure 1.5 are called *trees*; they are fundamental combinatorial structures that we shall encounter on numerous occasions throughout the book. We shall consider the properties of trees in detail in Chapter 5. For the *union* and *find* operations, the trees in Figure 1.5 are useful because they are quick to build and have the property that two objects are connected in the tree if and only if the objects are connected in the input. By moving up the tree, we can easily find the root of the tree containing each object, so we have a way to find whether or not they are connected. Each tree has precisely one object that points to itself, which is called the *root* of the tree. The self-pointer is not shown in the diagrams. When we start at any object in the tree, move to the object to which it points, then move to the object to which that object points, and so forth, we eventually end up at the root, always. We can prove this property to be true by induction: It is true after the array is initialized to have every object point to itself, and if it is true before a given *union* operation, it is certainly true afterward.

The diagrams in Figure 1.4 for the quick-find algorithm have the same properties as those described in the previous paragraph. The difference between the two is that we reach the root from all the nodes in the quick-find trees after following just one link, whereas we might need to follow several links to get to the root in a quick-union tree.

Program 1.2 is an implementation of the *union* and *find* operations that comprise the quick-union algorithm to solve the connectivity problem. The quick-union algorithm would seem to be faster than the

Program 1.2 Quick-union solution to connectivity problem

If we replace the body of the `while` loop in Program 1.1 by this code, we have a program that meets the same specifications as Program 1.1, but does less computation for the *union* operation at the expense of more computation for the *find* operation. The `for` loops and subsequent `if` statement in this code specify the necessary and sufficient conditions on the `id` array for `p` and `q` to be connected. The assignment statement `id[i] = j` implements the *union* operation.

```
for (i = p; i != id[i]; i = id[i]) ;
for (j = q; j != id[j]; j = id[j]) ;
if (i == j) continue;
id[i] = j;
printf(" %d %d\n", p, q);
```

quick-find algorithm, because it does not have to go through the entire array for each input pair; but how much faster is it? This question is more difficult to answer here than it was for quick find, because the running time is much more dependent on the nature of the input. By running empirical studies or doing mathematical analysis (see Chapter 2), we can convince ourselves that Program 1.2 is far more efficient than Program 1.1, and that it is feasible to consider using Program 1.2 for huge practical problems. We shall discuss one such empirical study at the end of this section. For the moment, we can regard quick union as an improvement because it removes quick find's main liability (that the program requires at least NM instructions to process M union operations among N objects).

This difference between quick union and quick find certainly represents an improvement, but quick union still has the liability that we cannot *guarantee* it to be substantially faster than quick find in every case, because the input data could conspire to make the *find* operation slow.

Property 1.2 *For $M > N$, the quick-union algorithm could take more than $MN/2$ instructions to solve a connectivity problem with M pairs of N objects.*

Suppose that the input pairs come in the order 1-2, then 2-3, then 3-4, and so forth. After $N - 1$ such pairs, we have N objects all in the same set, and the tree that is formed by the quick-union algorithm

<i>p</i>	<i>q</i>	0	1	2	3	4	5	6	7	8	9
3	4	0	1	2	4	4	5	6	7	8	9
4	9	0	1	2	4	9	5	6	7	8	9
8	0	0	1	2	4	9	5	6	7	0	9
2	3	0	1	9	4	9	5	6	7	0	9
5	6	0	1	9	4	9	6	6	7	0	9
2	9	0	1	9	4	9	6	6	7	0	9
5	9	0	1	9	4	9	6	9	7	0	9
7	3	0	1	9	4	9	6	9	9	0	9
4	8	0	1	9	4	9	6	9	9	0	0
5	6	0	1	9	4	9	6	9	9	0	0
0	2	0	1	9	4	9	6	9	9	0	0
6	1	1	1	9	4	9	6	9	9	0	0
5	8	1	1	9	4	9	6	9	9	0	0

Figure 1.6
Example of quick union (not-too-quick find)

This sequence depicts the contents of the `id` array after each of the pairs at left are processed by the quick-union algorithm (Program 1.2). Shaded entries are those that change for the union operation (just one per operation). When we process the pair `p q`, we follow pointers from `p` to get an entry `i` with `id[i] == i`; then, we follow pointers from `q` to get an entry `j` with `id[j] == j`; then, if `i` and `j` differ, we set `id[i] = id[j]`. For the find operation for the pair 5-8 (final line), `i` takes on the values 5 6 9 0 1, and `j` takes on the values 8 0 1.

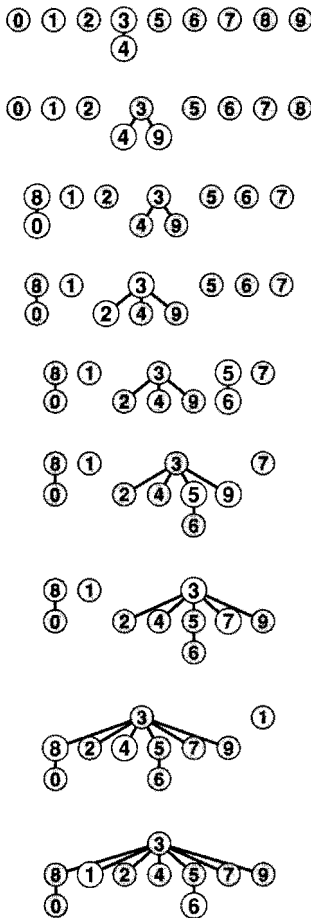


Figure 1.7
Tree representation of
weighted quick union

This sequence depicts the result of changing the quick-union algorithm to link the root of the smaller of the two trees to the root of the larger of the two trees. The distance from each node to the root of its tree is small, so the find operation is efficient.

is a straight line, with N pointing to $N - 1$, which points to $N - 2$, which points to $N - 3$, and so forth. To execute the *find* operation for object N , the program has to follow $N - 1$ pointers. Thus, the average number of pointers followed for the first N pairs is

$$(0 + 1 + \dots + (N - 1))/N = (N - 1)/2.$$

Now suppose that the remainder of the pairs all connect N to some other object. The *find* operation for each of these pairs involves at least $(N - 1)$ pointers. The grand total for the M *find* operations for this sequence of input pairs is certainly greater than $MN/2$. ■

Fortunately, there is an easy modification to the algorithm that allows us to guarantee that bad cases such as this one do not occur. Rather than arbitrarily connecting the second tree to the first for *union*, we keep track of the number of nodes in each tree and always connect the smaller tree to the larger. This change requires slightly more code and another array to hold the node counts, as shown in Program 1.3, but it leads to substantial improvements in efficiency. We refer to this algorithm as the *weighted quick-union algorithm*.

Figure 1.7 shows the forest of trees constructed by the weighted union–find algorithm for the example input in Figure 1.1. Even for this small example, the paths in the trees are substantially shorter than for the unweighted version in Figure 1.5. Figure 1.8 illustrates what happens in the worst case, when the sizes of the sets to be merged in the *union* operation are always equal (and a power of 2). These tree structures look complex, but they have the simple property that the maximum number of pointers that we need to follow to get to the root in a tree of 2^n nodes is n . Furthermore, when we merge two trees of 2^n nodes, we get a tree of 2^{n+1} nodes, and we increase the maximum distance to the root to $n + 1$. This observation generalizes to provide a proof that the weighted algorithm is substantially more efficient than the unweighted algorithm.

Property 1.3 *The weighted quick-union algorithm follows at most $2 \lg N$ pointers to determine whether two of N objects are connected.*

We can prove that the *union* operation preserves the property that the number of pointers followed from any node to the root in a set of k objects is no greater than $\lg k$ (we do not count the self-pointer at the root). When we combine a set of i nodes with a set of j nodes with

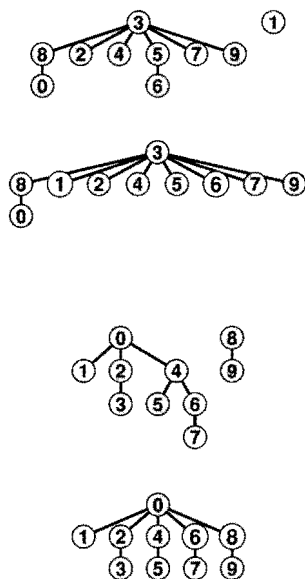


Figure 1.9
Path compression

We can make paths in the trees even shorter by simply making all the objects that we touch point to the root of the new tree for the union operation, as shown in these two examples. The example at the top shows the result corresponding to Figure 1.7. For short paths, path compression has no effect, but when we process the pair 1 6, we make 1, 5, and 6 all point to 3 and get a tree flatter than the one in Figure 1.7. The example at the bottom shows the result corresponding to Figure 1.8. Paths that are longer than one or two links can develop in the trees, but whenever we traverse them, we flatten them. Here, when we process the pair 6 8, we flatten the tree by making 4, 6, and 8 all point to 0.

literally millions of times faster than the simpler algorithms for the huge problems that we might encounter in practical applications.

It is evident from the diagrams that relatively few nodes are far from the root; indeed, empirical studies on huge problems tell us that the weighted quick-union algorithm of Program 1.3 typically can solve practical problems in *linear* time. That is, the cost of running the algorithm is within a constant factor of the cost of reading the input. We could hardly expect to find a more efficient algorithm.

We immediately come to the question of whether or not we can find an algorithm that has *guaranteed* linear performance. This question is an extremely difficult one that plagued researchers for many years (see Section 2.7). There are a number of easy ways to improve the weighted quick-union algorithm further. Ideally, we would like every node to point directly to the root of its tree, but we do not want to pay the price of changing a large number of pointers, as we did in the quick-union algorithm. We can approach the ideal simply by making all the nodes that we do examine point to the root. This step seems drastic at first blush, but it is easy to implement, and there is nothing sacrosanct about the structure of these trees: If we can modify them to make the algorithm more efficient, we should do so. We can implement this method, called *path compression*, easily, by adding another pass through each path during the *union* operation, setting the *id* entry corresponding to each vertex encountered along the way to point to the root. The net result is to flatten the trees almost completely, approximating the ideal achieved by the quick-find algorithm, as illustrated in Figure 1.9. The analysis that establishes this fact is extremely complex, but the method is simple and effective. Figure 1.11 shows the result of path compression for a large example.

There are many other ways to implement path compression. For example, Program 1.4 is an implementation that compresses the paths by making each link skip to the next node in the path on the way up the tree, as depicted in Figure 1.10. This method is slightly easier to implement than full path compression (see Exercise 1.16), and achieves the same net result. We refer to this variant as *weighted quick-union with path compression by halving*. Which of these methods is the more effective? Is the savings achieved worth the extra time required to implement path compression? Is there some other technique that we should consider? To answer these questions, we need to look more

Program 1.4 Path compression by halving

If we replace the for loops in Program 1.3 by this code, we halve the length of any path that we traverse. The net result of this change is that the trees become almost completely flat after a long sequence of operations.

```

for (i = p; i != id[i]; i = id[i])
    id[i] = id[id[i]];
for (j = q; j != id[j]; j = id[j])
    id[j] = id[id[j]];

```

carefully at the algorithms and implementations. We shall return to this topic in Chapter 2, in the context of our discussion of basic approaches to the analysis of algorithms.

The end result of the succession of algorithms that we have considered to solve the connectivity problem is about the best that we could hope for in any practical sense. We have algorithms that are easy to implement whose running time is guaranteed to be within a constant factor of the cost of gathering the data. Moreover, the algorithms are *online* algorithms that consider each edge once, using space proportional to the number of objects, so there is no limitation on the number of edges that they can handle. The empirical studies in Table 1.1 validate our conclusion that Program 1.3 and its path-compression variations are useful even for huge practical applications. Choosing which is the best among these algorithms requires careful and sophisticated analysis (see Chapter 2).

Exercises

- ▷ 1.4 Show the contents of the *id* array after each *union* operation when you use the quick-find algorithm (Program 1.1) to solve the connectivity problem for the sequence 0-2, 1-4, 2-5, 3-6, 0-4, 6-0, and 1-3. Also give the number of times the program accesses the *id* array for each input pair.
- ▷ 1.5 Do Exercise 1.4, but use the quick-union algorithm (Program 1.2).
- ▷ 1.6 Give the contents of the *id* array after each *union* operation for the weighted quick-union algorithm running on the examples corresponding to Figure 1.7 and Figure 1.8.
- ▷ 1.7 Do Exercise 1.4, but use the weighted quick-union algorithm (Program 1.3).

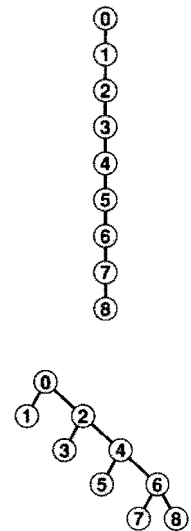


Figure 1.10
Path compression by halving

We can nearly halve the length of paths on the way up the tree by taking two links at a time, and setting the bottom one to point to the same node as the top one, as shown in this example. The net result of performing this operation on every path that we traverse is asymptotically the same as full path compression.

Table 1.1 Empirical study of union-find algorithms

These relative timings for solving random connectivity problems using various union-find algorithms demonstrate the effectiveness of the weighted version of the quick union algorithm. The added incremental benefit due to path compression is less important. In these experiments, M is the number of random connections generated until all N objects were connected. This process involves substantially more *find* operations than *union* operations, so quick union is substantially slower than quick find. Neither quick find nor quick union is feasible for huge N . The running time for the weighted methods is evidently proportional to N , as it approximately doubles when N is doubled.

N	M	F	U	W	P	H
1000	6206	14	25	6	5	3
2500	20236	82	210	13	15	12
5000	41913	304	1172	46	26	25
10000	83857	1216	4577	91	73	50
25000	309802			219	208	216
50000	708701			469	387	497
100000	1545119			1071	1106	1096

Key:

- F quick find (Program 1.1)
- U quick union (Program 1.2)
- W weighted quick union (Program 1.3)
- P weighted quick union with path compression (Exercise 1.16)
- H weighted quick union with halving (Program 1.4)

▷ 1.8 Do Exercise 1.4, but use the weighted quick-union algorithm with path compression by halving (Program 1.4).

1.9 Prove an upper bound on the number of machine instructions required to process M connections on N objects using Program 1.3. You may assume, for example, that any C assignment statement always requires less than c instructions, for some fixed constant c .

1.10 Estimate the minimum amount of time (in days) that would be required for quick find (Program 1.1) to solve a problem with 10^6 objects and 10^9 input pairs, on a computer capable of executing 10^9 instructions per second. Assume that each iteration of the `while` loop requires at least 10 instructions.

1.11 Estimate the maximum amount of time (in seconds) that would be required for weighted quick union (Program 1.3) to solve a problem with 10^6 objects and 10^9 input pairs, on a computer capable of executing 10^9 instructions per second. Assume that each iteration of the `while` loop requires at most 100 instructions.

1.12 Compute the *average* distance from a node to the root in a worst-case tree of 2^n nodes built by the weighted quick-union algorithm.

▷ **1.13** Draw a diagram like Figure 1.10, starting with eight nodes instead of nine.

○ **1.14** Give a sequence of input pairs that causes the weighted quick-union algorithm (Program 1.3) to produce a path of length 4.

● **1.15** Give a sequence of input pairs that causes the weighted quick-union algorithm with path compression by halving (Program 1.4) to produce a path of length 4.

1.16 Show how to modify Program 1.3 to implement *full* path compression, where we complete each *union* operation by making every node that we touch point to the root of the new tree.

▷ **1.17** Answer Exercise 1.4, but using the weighted quick-union algorithm with full path compression (Exercise 1.16).

●● **1.18** Give a sequence of input pairs that causes the weighted quick-union algorithm with full path compression (Exercise 1.16) to produce a path of length 4.

○ **1.19** Give an example showing that modifying quick union (Program 1.2) to implement full path compression (see Exercise 1.16) is not sufficient to ensure that the trees have no long paths.

● **1.20** Modify Program 1.3 to use the *height* of the trees (longest path from any node to the root), instead of the weight, to decide whether to set `id[i] = j` or `id[j] = i`. Run empirical studies to compare this variant with Program 1.3.

●● **1.21** Show that Property 1.3 holds for the algorithm described in Exercise 1.20.

● **1.22** Modify Program 1.4 to generate random pairs of integers between 0 and $N-1$ instead of reading them from standard input, and to loop until $N-1$ *union* operations have been performed. Run your program for $N = 10^3, 10^4, 10^5$, and 10^6 and print out the total number of edges generated for each value of N .

● **1.23** Modify your program from Exercise 1.22 to plot the number of edges needed to connect N items, for $100 \leq N \leq 1000$.

●● **1.24** Give an approximate formula for the number of random edges that are required to connect N objects, as a function of N .



Figure 1.11
A large example of the effect of path compression

This sequence depicts the result of processing random pairs from 100 objects with the weighted quick-union algorithm with path compression. All but two of the nodes in the tree are one or two steps from the root.

1.4 Perspective

Each of the algorithms that we considered in Section 1.3 seems to be an improvement over the previous in some intuitive sense, but the process is perhaps artificially smooth because we have the benefit of hindsight in looking over the development of the algorithms as they were studied by researchers over the years (*see reference section*). The implementations are simple and the problem is well specified, so we can evaluate the various algorithms directly by running empirical studies. Furthermore, we can validate these studies and quantify the comparative performance of these algorithms (*see Chapter 2*). Not all the problem domains in this book are as well developed as this one, and we certainly can run into complex algorithms that are difficult to compare and mathematical problems that are difficult to solve. We strive to make objective scientific judgements about the algorithms that we use, while gaining experience learning the properties of implementations running on actual data from applications or random test data.

The process is prototypical of the way that we consider various algorithms for fundamental problems throughout the book. When possible, we follow the same basic steps that we took for union-find algorithms in Section 1.2, some of which are highlighted in this list:

- Decide on a complete and specific problem statement, including identifying fundamental abstract operations that are intrinsic to the problem.
- Carefully develop a succinct implementation for a straightforward algorithm.
- Develop improved implementations through a process of stepwise refinement, validating the efficacy of ideas for improvement through empirical analysis, mathematical analysis, or both.
- Find high-level abstract representations of data structures or algorithms in operation that enable effective high-level design of improved versions.
- Strive for worst-case performance guarantees when possible, but accept good performance on actual data when available.

The potential for spectacular performance improvements for practical problems such as those that we saw in Section 1.2 makes algorithm design a compelling field of study; few other design activities hold the potential to reap savings factors of millions or billions, or more.

More important, as the scale of our computational power and our applications increases, the gap between a fast algorithm and a slow one grows. A new computer might be 10 times faster and be able to process 10 times as much data as an old one, but if we are using a quadratic algorithm such as quick find, the new computer will take 10 times as long on the new job as the old one took to finish the old job! This statement seems counterintuitive at first, but it is easily verified by the simple identity $(10N)^2/10 = 10N^2$, as we shall see in Chapter 2. As computational power increases to allow us to take on larger and larger problems, the importance of having efficient algorithms increases, as well.

Developing an efficient algorithm is an intellectually satisfying activity that can have direct practical payoff. As the connectivity problem indicates, a simply stated problem can lead us to study numerous algorithms that are not only both useful and interesting, but also intricate and challenging to understand. We shall encounter many ingenious algorithms that have been developed over the years for a host of practical problems. As the scope of applicability of computational solutions to scientific and commercial problems widens, so also grows the importance of being able to apply efficient algorithms to solve known problems and of being able to develop efficient solutions to new problems.

Exercises

1.25 Suppose that we use weighted quick union to process 10 times as many connections on a new computer that is 10 times as fast as an old one. How much longer would it take the new computer to finish the new job than it took the old one to finish the old job?

1.26 Answer Exercise 1.25 for the case where we use an algorithm that requires N^3 instructions.

1.5 Summary of Topics

This section comprises brief descriptions of the major parts of the book, giving specific topics covered and an indication of our general

orientation toward the material. This set of topics is intended to touch on as many fundamental algorithms as possible. Some of the areas covered are core computer-science areas that we study in depth to learn basic algorithms of wide applicability. Other algorithms that we discuss are from advanced fields of study within computer science and related fields, such as numerical analysis and operations research—in these cases, our treatment serves as an introduction to these fields through examination of basic methods.

The first four parts of the book, which are contained in this volume, cover the most widely used set of algorithms and data structures, a first level of abstraction for collections of objects with keys that can support a broad variety of important fundamental algorithms. The algorithms that we consider are the products of decades of research and development, and continue to play an essential role in the ever-expanding applications of computation.

Fundamentals (Part 1) in the context of this book are the basic principles and methodology that we use to implement, analyze, and compare algorithms. The material in Chapter 1 motivates our study of algorithm design and analysis; in Chapter 2, we consider basic methods of obtaining quantitative information about the performance of algorithms.

Data Structures (Part 2) go hand-in-hand with algorithms: we shall develop a thorough understanding of data representation methods for use throughout the rest of the book. We begin with an introduction to basic concrete data structures in Chapter 3, including arrays, linked lists, and strings; then we consider recursive programs and data structures in Chapter 5, in particular trees and algorithms for manipulating them. In Chapter 4, we consider fundamental abstract data types (ADTs) such as stacks and queues, including implementations using elementary data structures.

Sorting algorithms (Part 3) for rearranging files into order are of fundamental importance. We consider a variety of algorithms in considerable depth, including Shellsort, quicksort, mergesort, heapsort, and radix sorts. We shall encounter algorithms for several related problems, including priority queues, selection, and merging. Many of these algorithms will find application as the basis for other algorithms later in the book.

Searching algorithms (Part 4) for finding specific items among large collections of items are also of fundamental importance. We discuss basic and advanced methods for searching using trees and digital key transformations, including binary search trees, balanced trees, hashing, digital search trees and tries, and methods appropriate for huge files. We note relationships among these methods, comparative performance statistics, and correspondences to sorting methods.

Parts 5 through 8, which are contained in a separate volume, cover advanced applications of the algorithms described here for a diverse set of applications—a second level of abstractions specific to a number of important applications areas. We also delve more deeply into techniques of algorithm design and analysis. Many of the problems that we touch on are the subject of ongoing research.

String Processing algorithms (Part 5) include a range of methods for processing (long) sequences of characters. String searching leads to pattern matching, which leads to parsing. File-compression techniques are also considered. Again, an introduction to advanced topics is given through treatment of some elementary problems that are important in their own right.

Geometric Algorithms (Part 6) are methods for solving problems involving points and lines (and other simple geometric objects) that have only recently come into use. We consider algorithms for finding the convex hull of a set of points, for finding intersections among geometric objects, for solving closest-point problems, and for multidimensional searching. Many of these methods nicely complement the more elementary sorting and searching methods.

Graph Algorithms (Part 7) are useful for a variety of difficult and important problems. A general strategy for searching in graphs is developed and applied to fundamental connectivity problems, including shortest path, minimum spanning tree, network flow, and matching. A unified treatment of these algorithms shows that they are all based on the same procedure, and that this procedure depends on the basic priority queue ADT.

Advanced Topics (Part 8) are discussed for the purpose of relating the material in the book to several other advanced fields of study. We begin with major approaches to the design and analysis of algorithms, including divide-and-conquer, dynamic programming, randomization,

and amortization. We survey linear programming, the fast Fourier transform, NP-completeness, and other advanced topics from an introductory viewpoint to gain appreciation for the interesting advanced fields of study suggested by the elementary problems confronted in this book.

The study of algorithms is interesting because it is a new field (almost all the algorithms that we study are less than 50 years old, and some were just recently discovered) with a rich tradition (a few algorithms have been known for thousands of years). New discoveries are constantly being made, but few algorithms are completely understood. In this book we shall consider intricate, complicated, and difficult algorithms as well as elegant, simple, and easy algorithms. Our challenge is to understand the former and to appreciate the latter in the context of many different potential applications. In doing so, we shall explore a variety of useful tools and develop a style of algorithmic thinking that will serve us well in computational challenges to come.

CHAPTER TWO

Principles of Algorithm Analysis

ANALYSIS IS THE key to being able to understand algorithms sufficiently well that we can apply them effectively to practical problems. Although we cannot do extensive experimentation and deep mathematical analysis on each and every program that we run, we can work within a basic framework involving both empirical testing and approximate analysis that can help us to know the important facts about the performance characteristics of our algorithms, so that we may compare those algorithms and can apply them to practical problems.

The very idea of describing the performance of a complex algorithm accurately with a mathematical analysis seems a daunting prospect at first, and we do often call on the research literature for results based on detailed mathematical study. Although it is not our purpose in this book to cover methods of analysis or even to summarize these results, it is important for us to be aware at the outset that we are on firm scientific ground when we want to compare different methods. Moreover, a great deal of detailed information is available about many of our most important algorithms through careful application of relatively few elementary techniques. We do highlight basic analytic results and methods of analysis throughout the book, particularly when such understanding helps us to understand the inner workings of fundamental algorithms. Our primary goal in this chapter is to provide the context and the tools that we need to work intelligently with the algorithms themselves.

The example in Chapter 1 provides a context that illustrates many of the basic concepts of algorithm analysis, so we frequently refer

back to the performance of union-find algorithms to make particular points concrete. We also consider a detailed pair of new examples, in Section 2.6.

Analysis plays a role at every point in the process of designing and implementing algorithms. At first, as we saw, we can save factors of thousands or millions in the running time with appropriate algorithm design choices. As we consider more efficient algorithms, we find it more of a challenge to choose among them, so we need to study their properties in more detail. In pursuit of the *best* (in some precise technical sense) algorithm, we find both algorithms that are useful in practice and theoretical questions that are challenging to resolve.

Complete coverage of methods for the analysis of algorithms is the subject of a book in itself (*see reference section*), but it is worthwhile for us to consider the basics here, so that we can

- Illustrate the process.
- Describe in one place the mathematical conventions that we use.
- Provide a basis for discussion of higher-level issues.
- Develop an appreciation for scientific underpinnings of the conclusions that we draw when comparing algorithms.

Most important, algorithms and their analyses are often intertwined. In this book, we do not delve into deep and difficult mathematical derivations, but we do use sufficient mathematics to be able to understand what our algorithms are and how we can use them effectively.

2.1 Implementation and Empirical Analysis

We design and develop algorithms by layering abstract operations that help us to understand the essential nature of the computational problems that we want to solve. In theoretical studies, this process, although valuable, can take us far afield from the real-world problems that we need to consider. Thus, in this book, we keep our feet on the ground by expressing all the algorithms that we consider in an actual programming language: C. This approach sometimes leaves us with a blurred distinction between an algorithm and its implementation, but that is small price to pay for the ability to work with and to learn from a concrete implementation.

Indeed, carefully constructed programs in an actual programming language provide an effective means of expressing our algorithms.

In this book, we consider a large number of important and efficient algorithms that we describe in implementations that are both concise and precise in C. English-language descriptions or abstract high-level representations of algorithms are all too often vague or incomplete; actual implementations force us to discover economical representations to avoid being inundated in detail.

We express our algorithms in C, but this book is about algorithms, rather than about C programming. Certainly, we consider C implementations for many important tasks, and, when there is a particularly convenient or efficient way to do a task in C, we will take advantage of it. But the vast majority of the implementation decisions that we make are worth considering in any modern programming environment. Translating the programs in Chapter 1, and most of the other programs in this book, to another modern programming language is a straightforward task. On occasion, we also note when some other language provides a particularly effective mechanism suited to the task at hand. Our goal is to use C as a vehicle for expressing the algorithms that we consider, rather than to dwell on implementation issues specific to C.

If an algorithm is to be implemented as part of a large system, we use abstract data types or a similar mechanism to make it possible to change algorithms or implementations after we determine what part of the system deserves the most attention. From the start, however, we need to have an understanding of each algorithm's performance characteristics, because design requirements of the system may have a major influence on algorithm performance. Such initial design decisions must be made with care, because it often does turn out, in the end, that the performance of the whole system depends on the performance of some basic algorithm, such as those discussed in this book.

Implementations of the algorithms in this book have been put to effective use in a wide variety of large programs, operating systems, and applications systems. Our intention is to describe the algorithms and to encourage a focus on their dynamic properties through experimentation with the implementations given. For some applications, the implementations may be quite useful exactly as given; for other applications, however, more work may be required. For example, using a more defensive programming style than the one that we use in this

book is justified when we are building real systems. Error conditions must be checked and reported, and programs must be implemented such that they can be changed easily, read and understood quickly by other programmers, interface well with other parts of the system, and be amenable to being moved to other environments.

Notwithstanding all these comments, we take the position when analyzing each algorithm that performance is of critical importance, to focus our attention on the algorithm's essential performance characteristics. We assume that we are always interested in knowing about algorithms with substantially better performance, particularly if they are simpler.

To use an algorithm effectively, whether our goal is to solve a huge problem that could not otherwise be solved, or whether our goal is to provide an efficient implementation of a critical part of a system, we need to have an understanding of its performance characteristics. Developing such an understanding is the goal of algorithmic analysis.

One of the first steps that we take to understand the performance of algorithms is to do *empirical analysis*. Given two algorithms to solve the same problem, there is no mystery in the method: We run them both to see which one takes longer! This concept might seem too obvious to mention, but it is an all-too-common omission in the comparative study of algorithms. The fact that one algorithm is 10 times faster than another is unlikely to escape the notice of someone who waits 3 seconds for one to finish and 30 seconds for the other to finish, but it is easy to overlook as a small constant overhead factor in a mathematical analysis. When we monitor the performance of careful implementations on typical input, we get performance results that not only give us a direct indicator of efficiency, but also provide us with the information that we need to compare algorithms and to validate any mathematical analyses that may apply (see, for example, Table 1.1). When empirical studies start to consume a significant amount of time, mathematical analysis is called for. Waiting an hour or a day for a program to finish is hardly a productive way to find out that it is slow, particularly when a straightforward analysis can give us the same information.

The first challenge that we face in empirical analysis is to develop a correct and complete implementation. For some complex algorithms, this challenge may present a significant obstacle. Accordingly, we

typically want to have, through analysis or through experience with similar programs, some indication of how efficient a program might be before we invest too much effort in getting it to work.

The second challenge that we face in empirical analysis is to determine the nature of the input data and other factors that have direct influence on the experiments to be performed. Typically, we have three basic choices: use *actual* data, *random* data, or *perverse* data. Actual data enable us truly to measure the cost of the program in use; random data assure us that our experiments test the algorithm, not the data; and perverse data assure us that our programs can handle any input presented them. For example, when we test sorting algorithms, we run them on data such as the words in *Moby Dick*, on randomly generated integers, and on files of numbers that are all the same value. This problem of determining which input data to use to compare algorithms also arises when we analyze the algorithms.

It is easy to make mistakes when we compare implementations, particularly if differing machines, compilers, or systems are involved, or if huge programs with ill-specified inputs are being compared. The principal danger in comparing programs empirically is that one implementation may be coded more carefully than the other. The inventor of a proposed new algorithm is likely to pay careful attention to every aspect of its implementation, and not to expend so much effort on the details of implementing a classical competing algorithm. To be confident of the accuracy of an empirical study comparing algorithms, we must be sure to give the same attention to each implementation.

One approach that we often use in this book, as we saw in Chapter 1, is to derive algorithms by making relatively minor modifications to other algorithms for the same problem, so comparative studies really are valid. More generally, we strive to identify essential abstract operations, and start by comparing algorithms on the basis of their use of such operations. For example, the comparative empirical results that we examined in Table 1.1 are likely to be robust across programming languages and environments, as they involve programs that are similar and that make use of the same set of basic operations. For a particular programming environment, we can easily relate these numbers to actual running times. Most often, we simply want to know which of two programs is likely to be faster, or to what extent a certain change will improve the time or space requirements of a certain program.

Choosing among algorithms to solve a given problem is tricky business. Perhaps the most common mistake made in selecting an algorithm is to ignore performance characteristics. Faster algorithms are often more complicated than brute-force solutions, and implementors are often willing to accept a slower algorithm to avoid having to deal with added complexity. As we saw with union-find algorithms, however, we can sometimes reap huge savings with just a few lines of code. Users of a surprising number of computer systems lose substantial time waiting for simple quadratic algorithms to finish when $N \log N$ algorithms are available that are only slightly more complicated and could run in a fraction of the time. When we are dealing with huge problem sizes, we have no choice but to seek a better algorithm, as we shall see.

Perhaps the second most common mistake made in selecting an algorithm is to pay too much attention to performance characteristics. Improving the running time of a program by a factor of 10 is inconsequential if the program takes only a few microseconds. Even if a program takes a few minutes, it may not be worth the time and effort required to make it run 10 times faster, particularly if we expect to use the program only a few times. The total time required to implement and debug an improved algorithm might be substantially more than the time required simply to run a slightly slower one—we may as well let the computer do the work. Worse, we may spend a considerable amount of time and effort implementing ideas that should improve a program but actually do not do so.

We cannot run empirical tests for a program that is not yet written, but we can analyze properties of the program and estimate the potential effectiveness of a proposed improvement. Not all putative improvements actually result in performance gains, and we need to understand the extent of the savings realized at each step. Moreover, we can include parameters in our implementations, and can use analysis to help us set the parameters. Most important, by understanding the fundamental properties of our programs and the basic nature of the programs' resource usage, we hold the potentials to evaluate their effectiveness on computers not yet built and to compare them against new algorithms not yet designed. In Section 2.2, we outline our methodology for developing a basic understanding of algorithm performance.

Exercises

2.1 Translate the programs in Chapter 1 to another programming language, and answer Exercise 1.22 for your implementations.

2.2 How long does it take to count to 1 billion (ignoring overflow)? Determine the amount of time it takes the program

```
int i, j, k, count = 0;
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
        for (k = 0; k < N; k++)
            count++;
```

to complete in your programming environment, for $N = 10$, 100, and 1000. If your compiler has optimization features that are supposed to make programs more efficient, check whether or not they do so for this program.

2.2 Analysis of Algorithms

In this section, we outline the framework within which mathematical analysis can play a role in the process of comparing the performance of algorithms, to lay a foundation for us to be able to consider basic analytic results as they apply to the fundamental algorithms that we consider throughout the book. We shall consider the basic mathematical tools that are used in the analysis of algorithms, both to allow us to study classical analyses of fundamental algorithms and to make use of results from the research literature that help us understand the performance characteristics of our algorithms.

The following are among the reasons that we perform mathematical analysis of algorithms:

- To compare different algorithms for the same task
- To predict performance in a new environment
- To set values of algorithm parameters

We shall see many examples of each of these reasons throughout the book. Empirical analysis might suffice for some of these tasks, but mathematical analysis can be more informative (and less expensive!), as we shall see.

The analysis of algorithms can be challenging indeed. Some of the algorithms in this book are well understood, to the point that accurate mathematical formulas are known that can be used to predict running time in practical situations. People develop such formulas by carefully studying the program, to find the running time in terms of fundamental

mathematical quantities, and then doing a mathematical analysis of the quantities involved. On the other hand, the performance properties of other algorithms in this book are not fully understood—perhaps their analysis leads to unsolved mathematical questions, or perhaps known implementations are too complex for a detailed analysis to be reasonable, or (most likely) perhaps the types of input that they encounter cannot be characterized accurately.

Several important factors in a precise analysis are usually outside a given programmer's domain of influence. First, C programs are translated into machine code for a given computer, and it can be a challenging task to figure out exactly how long even one C statement might take to execute (especially in an environment where resources are being shared, so even the same program can have varying performance characteristics at two different times). Second, many programs are extremely sensitive to their input data, and performance might fluctuate wildly depending on the input. Third, many programs of interest are not well understood, and specific mathematical results may not be available. Finally, two programs might not be comparable at all: one may run much more efficiently on one particular kind of input, the other runs efficiently under other circumstances.

All these factors notwithstanding, it is often possible to predict precisely how long a particular program will take, or to know that one program will do better than another in particular situations. Moreover, we can often acquire such knowledge by using one of a relatively small set of mathematical tools. It is the task of the algorithm analyst to discover as much information as possible about the performance of algorithms; it is the task of the programmer to apply such information in selecting algorithms for particular applications. In this and the next several sections, we concentrate on the idealized world of the analyst. To make effective use of our best algorithms, we need to be able to step into this world, on occasion.

The first step in the analysis of an algorithm is to identify the abstract operations on which the algorithm is based, to separate the analysis from the implementation. Thus, for example, we separate the study of how many times one of our *union-find* implementations executes the code fragment `i = a[i]` from the analysis of how many nanoseconds might be required to execute that particular code fragment on our computer. We need both these elements to determine

the actual running time of the program on a particular computer. The former is determined by properties of the algorithm; the latter by properties of the computer. This separation often allows us to compare algorithms in a way that is independent of particular implementations or of particular computers.

Although the number of abstract operations involved can be large, in principle, the performance of an algorithm typically depends on only a few quantities, and typically the most important quantities to analyze are easy to identify. One way to identify them is to use a profiling mechanism (a mechanism available in many C implementations that gives instruction-frequency counts) to determine the most frequently executed parts of the program for some sample runs. Or, like the union-find algorithms of Section 1.3, our implementation might be built on a few abstract operations. In either case, the analysis amounts to determining the frequency of execution of a few fundamental operations. Our *modus operandi* will be to look for rough estimates of these quantities, secure in the knowledge that we can undertake a fuller analysis for important programs when necessary. Moreover, as we shall see, we can often use approximate analytic results in conjunction with empirical studies to predict performance accurately.

We also have to study the data, and to model the input that might be presented to the algorithm. Most often, we consider one of two approaches to the analysis: we either assume that the input is random, and study the *average-case* performance of the program, or we look for perverse input, and study the *worst-case* performance of the program. The process of characterizing random inputs is difficult for many algorithms, but for many other algorithms it is straightforward and leads to analytic results that provide useful information. The average case might be a mathematical fiction that is not representative of the data on which the program is being used, and the worst case might be a bizarre construction that would never occur in practice, but these analyses give useful information on performance in most cases. For example, we can test analytic results against empirical results (see Section 2.1). If they match, we have increased confidence in both; if they do not match, we can learn about the algorithm and the model by studying the discrepancies.

In the next three sections, we briefly survey the mathematical tools that we shall be using throughout the book. This material is

outside our primary narrative thrust, and readers with a strong background in mathematics or readers who are not planning to check our mathematical statements on the performance of algorithms in detail may wish to skip to Section 2.6 and to refer back to this material when warranted later in the book. The mathematical underpinnings that we consider, however, are generally not difficult to comprehend, and they are too close to core issues of algorithm design to be ignored by anyone wishing to use a computer effectively.

First, in Section 2.3, we consider the mathematical functions that we commonly need to describe the performance characteristics of algorithms. Next, in Section 2.4, we consider the *O-notation*, and the notion of *is proportional to*, which allow us to suppress detail in our mathematical analyses. Then, in Section 2.5, we consider *recurrence relations*, the basic analytic tool that we use to capture the performance characteristics of an algorithm in a mathematical equation. Following this survey, we consider examples where we use the basic tools to analyze specific algorithms, in Section 2.6.

Exercises

- **2.3** Develop an expression of the form $c_0 + c_1N + c_2N^2 + c_3N^3$ that accurately describes the running time of your program from Exercise 2.2. Compare the times predicted by this expression with actual times, for $N = 10, 100$, and 1000 .
- **2.4** Develop an expression that accurately describes the running time of Program 1.1 in terms of M and N .

2.3 Growth of Functions

Most algorithms have a *primary parameter* N that affects the running time most significantly. The parameter N might be the degree of a polynomial, the size of a file to be sorted or searched, the number of characters in a text string, or some other abstract measure of the size of the problem being considered: it is most often directly proportional to the size of the data set being processed. When there is more than one such parameter (for example, M and N in the *union-find* algorithms that we discussed in Section 1.3), we often reduce the analysis to just one parameter by expressing one of the parameters as a function of the other or by considering one parameter at a time (holding the other constant), so we can restrict ourselves to considering a single parameter

N without loss of generality. Our goal is to express the resource requirements of our programs (most often running time) in terms of N , using mathematical formulas that are as simple as possible and that are accurate for large values of the parameters. The algorithms in this book typically have running times proportional to one of the following functions:

- 1 Most instructions of most programs are executed once or at most only a few times. If all the instructions of a program have this property, we say that the program's running time is *constant*.
- $\log N$ When the running time of a program is *logarithmic*, the program gets slightly slower as N grows. This running time commonly occurs in programs that solve a big problem by transformation into a series of smaller problems, cutting the problem size by some constant fraction at each step. For our range of interest, we can consider the running time to be less than a large constant. The base of the logarithm changes the constant, but not by much: When N is 1 thousand, $\log N$ is 3 if the base is 10, or is about 10 if the base is 2; when N is 1 million, $\log N$ is only double these values. Whenever N doubles, $\log N$ increases by a constant, but $\log N$ does not double until N increases to N^2 .
- N When the running time of a program is *linear*, it is generally the case that a small amount of processing is done on each input element. When N is 1 million, then so is the running time. Whenever N doubles, then so does the running time. This situation is optimal for an algorithm that must process N inputs (or produce N outputs).
- $N \log N$ The $N \log N$ running time arises when algorithms solve a problem by breaking it up into smaller subproblems, solving them independently, and then combining the solutions. For lack of a better adjective (*linearithmic?*), we simply say that the running time of such an algorithm is $N \log N$. When N is 1 million, $N \log N$ is perhaps 20 million. When N doubles, the running time more (but not much more) than doubles.

seconds

10^2	1.7 minutes
10^4	2.8 hours
10^5	1.1 days
10^6	1.6 weeks
10^7	3.8 months
10^8	3.1 years
10^9	3.1 decades
10^{10}	3.1 centuries
10^{11}	never

Figure 2.1
Seconds conversions

The vast difference between numbers such as 10^4 and 10^8 is more obvious when we consider them to measure time in seconds and convert to familiar units of time. We might let a program run for 2.8 hours, but we would be unlikely to contemplate running a program that would take at least 3.1 years to complete. Because 2^{10} is approximately 10^3 , this table is useful for powers of 2 as well. For example, 2^{32} seconds is about 124 years.

N^2 When the running time of an algorithm is *quadratic*, that algorithm is practical for use on only relatively small problems. Quadratic running times typically arise in algorithms that process all pairs of data items (perhaps in a double nested loop). When N is 1 thousand, the running time is 1 million. Whenever N doubles, the running time increases fourfold.

N^3 Similarly, an algorithm that processes triples of data items (perhaps in a triple-nested loop) has a *cubic* running time and is practical for use on only small problems. When N is 100, the running time is 1 million. Whenever N doubles, the running time increases eightfold.

2^N Few algorithms with *exponential* running time are likely to be appropriate for practical use, even though such algorithms arise naturally as brute-force solutions to problems. When N is 20, the running time is 1 million. Whenever N doubles, the running time squares!

The running time of a particular program is likely to be some constant multiplied by one of these terms (the *leading term*) plus some smaller terms. The values of the constant coefficient and the terms included depend on the results of the analysis and on implementation details. Roughly, the coefficient of the leading term has to do with the number of instructions in the inner loop: At any level of algorithm design, it is prudent to limit the number of such instructions. For large N , the effect of the leading term dominates; for small N or for carefully engineered algorithms, more terms may contribute and comparisons of algorithms are more difficult. In most cases, we will refer to the running time of programs simply as “linear,” “ $N \log N$,” “cubic,” and so forth. We consider the justification for doing so in detail in Section 2.4.

Eventually, to reduce the total running time of a program, we focus on minimizing the number of instructions in the inner loop. Each instruction comes under scrutiny: Is it really necessary? Is there a more efficient way to accomplish the same task? Some programmers believe that the automatic tools provided by modern compilers can produce the best machine code; others believe that the best route is to hand-code inner loops into machine or assembly language. We normally

Table 2.1 Time to solve huge problems

For many applications, our only chance to be able to solve huge problem instances is to use an efficient algorithm. This table indicates the minimum amount of time required to solve problems of size 1 million and 1 billion, using linear, $N \lg N$, and quadratic algorithms, on computers capable of executing 1 million, 1 billion, and 1 trillion instructions per second. A fast algorithm enables us to solve a problem on a slow machine, but a fast machine is no help when we are using a slow algorithm.

operations per second	problem size 1 million			problem size 1 billion		
	N	$N \lg N$	N^2	N	$N \lg N$	N^2
10^6	seconds	seconds	weeks	hours	hours	never
10^9	instant	instant	hours	seconds	seconds	decades
10^{12}	instant	instant	seconds	instant	instant	weeks

stop short of considering optimization at this level, although we do occasionally take note of how many machine instructions are required for certain operations, to help us understand why one algorithm might be faster than another in practice.

For small problems, it makes scant difference which method we use—a fast modern computer will complete the job in an instant. But as problem size increases, the numbers we deal with can become huge, as indicated in Table 2.2. As the number of instructions to be executed by a slow algorithm becomes truly huge, the time required to execute those instructions becomes infeasible, even for the fastest computers. Figure 2.1 gives conversion factors from large numbers of seconds to days, months, years, and so forth; Table 2.1 gives examples showing how fast algorithms are more likely than fast computers to be able to help us solve problems without facing outrageous running times.

A few other functions do arise. For example, an algorithm with N^2 inputs that has a running time proportional to N^3 is best thought of as an $N^{3/2}$ algorithm. Also, some algorithms have two stages of subproblem decomposition, which lead to running times proportional to $N \log^2 N$. It is evident from Table 2.2 that both of these functions are much closer to $N \log N$ than to N^2 .

Table 2.2 Values of commonly encountered functions

This table indicates the relative size of some of the functions that we encounter in the analysis of algorithms. The quadratic function clearly dominates, particularly for large N , and differences among smaller functions may not be as we might expect for small N . For example, $N^{3/2}$ should be greater than $N \lg^2 N$ for huge values of N , but $N \lg^2 N$ is greater for the smaller values of N that might occur in practice. A precise characterization of the running time of an algorithm might involve linear combinations of these functions. We can easily separate fast algorithms from slow ones because of vast differences between, for example, $\lg N$ and N or N and N^2 , but distinguishing among fast algorithms involves careful study.

$\lg N$	$\sqrt{N}$	N	$N \lg N$	$N(\lg N)^2$	$N^{3/2}$	N^2
3	3	10	33	110	32	100
7	10	100	664	4414	1000	10000
10	32	1000	9966	99317	31623	1000000
13	100	10000	132877	1765633	1000000	100000000
17	316	100000	1660964	27588016	31622777	10000000000
20	1000	1000000	19931569	397267426	1000000000	1000000000000

The logarithm function plays a special role in the design and analysis of algorithms, so it is worthwhile for us to consider it in detail. Because we often deal with analytic results only to within a constant factor, we use the notation “ $\log N$ ” without specifying the base. Changing the base from one constant to another changes the value of the logarithm by only a constant factor, but specific bases normally suggest themselves in particular contexts. In mathematics, the *natural logarithm* (base $e = 2.71828 \dots$) is so important that a special abbreviation is commonly used: $\log_e N \equiv \ln N$. In computer science, the *binary logarithm* (base 2) is so important that the abbreviation $\log_2 N \equiv \lg N$ is commonly used.

Occasionally, we iterate the logarithm: We apply it successively to a huge number. For example, $\lg \lg 2^{256} = \lg 256 = 8$. As illustrated by this example, we generally regard $\log \log N$ as a constant, for practical purposes, because it is so small, even when N is huge.

The smallest integer larger than $\lg N$ is the number of bits required to represent N in binary, in the same way that the smallest integer larger than $\log_{10} N$ is the number of digits required to represent N in decimal. The C statement

```
for (lgN = 0; N > 0; lgN++, N /= 2) ;
```

is a simple way to compute the smallest integer larger than $\lg N$. A similar method for computing this function is

```
for (lgN = 0, t = 1; t < N; lgN++, t += t) ;
```

This version emphasizes that $2^n \leq N < 2^{n+1}$ when n is the smallest integer larger than $\lg N$.

We also frequently encounter a number of special functions and mathematical notations from classical analysis that are useful in providing concise descriptions of properties of programs. Table 2.3 summarizes the most familiar of these functions; we briefly discuss them and some of their most important properties in the following paragraphs.

Our algorithms and analyses most often deal with discrete units, so we often have need for the following special functions to convert real numbers to integers:

$\lfloor x \rfloor$: largest integer less than or equal to x

$\lceil x \rceil$: smallest integer greater than or equal to x .

For example, $\lfloor \pi \rfloor$ and $\lceil e \rceil$ are both equal to 3, and $\lceil \lg(N+1) \rceil$ is the number of bits in the binary representation of N . Another important use of these functions arises when we want to divide a set of N objects in half. We cannot do so exactly if N is odd, so, to be precise, we divide into one subset with $\lfloor N/2 \rfloor$ objects and another subset with $\lceil N/2 \rceil$ objects. If N is even, the two subsets are equal in size ($\lfloor N/2 \rfloor = \lceil N/2 \rceil$); if N is odd, they differ in size by 1 ($\lfloor N/2 \rfloor + 1 = \lceil N/2 \rceil$). In C, we can compute these functions directly when we are operating on integers (for example, if $N \geq 0$, then $N/2$ is $\lfloor N/2 \rfloor$ and $N - (N/2)$ is $\lceil N/2 \rceil$), and we can use `floor` and `ceil` from `math.h` to compute them when we are operating on floating point numbers.

A discretized version of the natural logarithm function called the *harmonic numbers* often arises in the analysis of algorithms. The N th harmonic number is defined by the equation

$$H_N = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{N}.$$